

EXPO THE EXPLORER • RUNTIME IMPLEMENTATION SPEC • REV
2026-08-30

Day Runtime Specification

Harici bir web tabanlı Day Editor ve balancing simulator yazmak için gereken davranış sözleşmesi. Her madde Unity kaynağından doğrulanmıştır; sınıf ve metod adları her bölümde verilir.

kaynak: `Assets/Scripts`

Unity 6000.x

JsonUtility

UNKNOWN işaretli

§0 Kapsam ve okuma notu

Bu döküman Day Editor kullanım kılavuzunun devamı değil, tamamlayıcısıdır: burada UI değil, çalışan kodun davranışı tarif edilir.

Her iddianın altında kaynak çıpası vardır — `ClassName.MethodName` — ve iddia yalnızca o dosyadan okunarak yazılmıştır. Kodda göremediğim hiçbir davranış varsayılmamıştır.

İŞARETLEME

UNKNOWN = kodda kesin cevabı bulunmayan, veya kasten belirtilmemiş (unspecified) davranış. **NEEDS DECISION** = simülatörü yazarken senin bir karar vermen gereken nokta. Tam liste §11'de toplanmıştır.

SİSTEMİN GENEL ŞEKLİ

İki tamamen ayrı rastgelelik alanı var ve bunları karıştırmamak simülatörün en kritik tasarım kararıdır:

ALAN	NE ZAMAN ÇALIŞIR	GİRDİSİ	ÇIKTISI
Authoring	Editörde "Generate" tıklanınca	<code>editorMeta.ticketGeneration</code> + food pool + seed	<code>runtime.ticketSequence</code> (diske yazılır)
Runtime	Oyun oynanırken, her ticket atamasında	<code>runtime.boardDistribution</code> + <code>runtime.ticketRuntime</code> + canlı state	Tahtaya düşen item'lar (kalıcı değil)

Authoring bir kez çalışır ve sonucu JSON'a gömülür. Runtime her oyunda yeniden çalışır ve hiçbir yere yazılmaz. Bir balancing simulator'ın asıl işi ikincisidir; birincisi yalnızca girdi üretmek için gerekir.

§1 Day JSON şeması

Tam C# model ağacı, tipler, default'lar, opsiyonellik ve eksik alan davranışı.

1.1 Serialization sistemi

`DayCatalogParser.ParseAll` `DayJsonSource.LoadAll`

`UnityEngine.JsonUtility` kullanılıyor — üçüncü parti JSON kütüphanesi yok. Yükleme zinciri:

```
// 1. Klasörün tamamı TextAsset olarak okunur
Resources.LoadAll<TextAsset>("Days") // DayJsonSource
// 2. Her dosya ayrı ayrı parse edilir
JsonUtility.FromJson<DayJson>(file.Json) // try/catch içinde
// 3. Resolve: id'ler FoodCatalog'a, enum'lar string'den
// 4. Sonuç DayIndex'e göre sıralanır
results.Sort((a, b) => a.DayIndex.CompareTo(b.DayIndex));
```

Bunun simülatör için üç önemli sonucu var:

- **Dosya adı hiçbir şey belirlemez.** Oyun sırası `runtime.dayIndex` alanından gelir; `day_07.json` içinde `dayIndex: 3` yazıyorsa o gün 3. sıradadır. Dosya adı yalnızca Day Editor'ün kendi kaydetme/yeniden adlandırma mantığında kullanılır.
- **Bir günün bozuk olması diğerlerini etkilemez.** Parse veya resolve hatası o günü listeden düşürür, `Debug.LogError` basar, döngü devam eder.
- **Yinelenen `dayIndex` hata basar ama günü düşürmez** — her ikisi de listeye eklenir.

`DayCatalogParser.ParseAll` · `seenIndices`

UNKNOWN — JSONUTILITY EKSİK-ALAN SEMANTIĞİ

Kod içi yorumlar (`DayJson.cs`, `DayCatalogParser.ResolveBoardDistribution`) şunu *iddia ediyor*: iç içe `[Serializable]` bir sınıf alanı JSON'da yoksa, `JsonUtility` null değil, tüm alanları CLR default'unda olan bir örnek döndürür. Bütün "absence by content" tasarımı bu iddiaya dayanıyor.

Buna karşın `ResolveDay` yine de `runtime == null` kontrolü yapıyor — yani kodun kendisi iki ihtimale karşı da korunmuş. Ben bu davranışı çalıştırıp doğrulamadım. **NEEDS DECISION**: TypeScript tarafında `JSON.parse` eksik alanı `undefined` bırakır. Simülatörün, `parse`'tan hemen sonra bir **normalizasyon katmanı** çalıştırıp eksik nesneleri "tüm alanları sıfır" nesnelere çevirmesi gerekir — yoksa aşağıdaki absence kuralları hiç tetiklenmez.

Diziler farklı: kod her yerde `?? Array.Empty<T>()` ile korunuyor, yani **eksik dizi null olarak gelebilir** ve bu beklenen durumdur.

1.2 Kök model ağacı

Systems/DaySystem/DayJson.cs

```

DayJson
├─ runtime      : DayRuntimeJson    ← yoksa gün düşer
└─ editorMeta   : DayEditorMetaJson ← runtime hiç okumaz

DayRuntimeJson
├─ dayIndex      : int
├─ ticketsRequiredForDay : int
├─ boardDistribution : BoardDistributionJson ← yoksa gün düşer
├─ ticketRuntime   : TicketRuntimeJson    ← yoksa gün düşer
├─ ticketSequence  : TicketEntryJson[]
├─ boardTimeline   : BoardSpawnEntryJson[]
└─ tutorial        : TutorialJson         ← opsiyonel

DayEditorMetaJson
├─ allowedFoodItemIds : string[]
└─ ticketGeneration   : TicketGenerationJson

```

editorMeta runtime tarafından hiçbir koşulda okunmaz. DayCatalogParser içinde tek bir referansı yoktur — bu sözleşme kod yorumunda açıkça yazılıdır. Simülatörde de bu ayrımı koru: editorMeta yalnızca generateTicketSequence 'ın girdisidir.

1.3 Alan tabloları

DAYRUNTIMEJSON

ALAN	C# TIPI	DEFAULT	ZORUNLU?	NOT
dayIndex	int	0	de facto	Sıralama anahtarı. 0 geçerli bir değer, kontrol edilmez.
ticketsRequiredForDay	int	0	de facto	Parser kontrol etmez; yalnız <code>DayValidator</code> sequence uzunluğuyla karşılaştırır.
boardDistribution	object	—	zorunlu	Yoksa/geçersizse gün tamamen düşer .
ticketRuntime	object	—	zorunlu	Yoksa/geçersizse gün tamamen düşer .
ticketSequence	array	null → []	opsiyonel	Boş dizi parse edilir; gün 0 biletle yüklenir.
boardTimeline	array	null → []	opsiyonel	Boş = açılış tahtası yok.
tutorial	object	enabled=false	opsiyonel	Bozuk olsa bile gün düşmez, sadece tutorial null olur.

BOARDDISTRIBUTIONJSON

`DayCatalogParser.ResolveBoardDistribution` `BoardDistributionSettings` ctor

ALAN	TIP	PARSER REDDİ	CTOR CLAMP	EDITÖR ARALIĞI
noiseLeakCountLambda	float	—	yok	0–10
guaranteedTicketCountMode	string	boş veya parse edilemez → gün düşer	—	Manual / Poisson
guaranteedTicketCount	int	< 1 → gün düşer	Clamp(1,3)	1–3
guaranteedTicketCountLambda	float	—	yok	0–10
earlyTicketWeightDecay	float	—	Clamp01	0–1
urgentTimeThresholdSeconds	float	—	Max(0)	0–30
leakDepth	int	< 1 → gün düşer	Clamp(1,10)	1–10
maxLeakCount	int	< 1 → gün düşer	Clamp(1,10)	1–10

Absence tespiti içerikten yapılır: `guaranteedTicketCountMode` boş string ise blok "yazılmamış" sayılır ve gün düşer. İki lambda kasten clamp'lenmez — bunlar Poisson oranıdır ve `TruncatedPoisson` sonucu zaten sınırlar.

TICKETRUNTIMEJSON

`DayCatalogParser.ResolveTicketRuntime`

`TicketRuntimeSettings` ctor

ALAN	TIP	PARSER REDDI	CTOR CLAMP
<code>impatientTimeLimitSeconds</code>	float	<code><= 0</code> → gün düşer	Max(0)
<code>normalTimeLimitSeconds</code>	float	<code><= 0</code> → gün düşer	Max(0)
<code>patientTimeLimitSeconds</code>	float	<code><= 0</code> → gün düşer	Max(0)
<code>upcomingQueueSize</code>	int	<code>< 1</code> → gün düşer	Max(1)

Burada absence tespiti üç sürenin de `> 0` olmasını şart koşarak yapılır — yarı dolu bir blok tek alan üzerinden geçemez.

TICKETENTRYJSON

`DayCatalogParser.ResolveTicketEntry`

ALAN	TIP	ZORUNLU?	ÇÖZÜMLENEMEZSE
<code>mainItemId</code>	string	zorunlu	Katalogda yoksa gün düşer
<code>sideItemId</code>	string	opsiyonel	Boş → null. Doluysa ve yoksa gün düşer
<code>drinkItemId</code>	string	opsiyonel	Boş → null. Doluysa ve yoksa gün düşer
<code>modifications</code>	array	opsiyonel	Bilinmeyen id → gün düşer
<code>patienceType</code>	string	zorunlu	Parse edilemezse gün düşer
<code>customerNameOverride</code>	string	opsiyonel	Boş → isim havuzundan rastgele
<code>timeLimitSecondsOverride</code>	float	opsiyonel	<code>0</code> → kapalı (bkz. §8.1)

BOARDSPAWNENTRYJSON · MODIFICATIONENTRYJSON

ALAN	TIP	ANLAMI
triggerStepIndex	int	-1 = Day Start. Başka değerler şemada var ama hiçbir yerden tetiklenmiyor – bkz. §7.
itemId	string	Katalogda yoksa gün düşer .
modifications	array	Item'ın taşıdığı modifikasyonlar.
useExactCell	bool	false → boş hücreye yerleştirilir (deterministik, bkz. §7.2).
x, y	int	Yalnız useExactCell true iken anlamlı.
modificationId	string	Modifikasyon id'si.
isAddition	bool	true = ekle, false = çıkar.

TICKETGENERATIONJSON · MAINDISHWEIGHTJSON (EDITORMETA)

ALAN	TIP	EKSIKSE
sideInclusionChance	float	0.0
drinkInclusionChance	float	0.0
modificationCountLambda	float	0.0 – pratikte okunmaz
modificationAdditionChance	float	0.0
patientTicketCount	int	0
normalTicketCount	int	0
impatientTicketCount	int	0
mainDishWeights[]	array	null → []
· foodItemId	string	Katalogda yoksa satır sessizce düşer
· weight	float	0.0
· modificationCountLambda	float	0.0
· maxModificationCount	int	0 → 10'a normalize edilir

KRİTİK: MAXMODIFICATIONCOUNT = 0

`MainDishWeight.NormalizeMaxModificationCount`

Bu alan sonradan eklendiği için eski dosyalarda yok ve 0 olarak deserializ oluyor. 0 düz okunsa "bu yemek asla modifikasyon almaz" demek olurdu — sessiz bir balans değişikliği. Bu yüzden **okuma anında** normalize edilir:

```
NormalizeMaxModificationCount(v) => v < 1 ? 10 : v
```

Simülâtörde bu kuralı *tek bir yerde* tut. Editör slider'ı da aynı fonksiyondan geçiyor.

1.4 Enum'lar ve geçerli değerler

Üç enum JSON'da **string** olarak tutulur ve `Enum.TryParse` ile çözülür. `TryParse` burada `ignoreCase` parametresi olmadan çağrılıyor, yani **büyük/küçük harf duyarlıdır**: "patient" parse edilmez, gün düşer.

ENUM	JSON ALANI	GEÇERLİ DEĞERLER	ORDINAL
PatienceType	patienceType	Impatient · Normal · Patient	0 · 1 · 2
GuaranteedTicketCountMode	guaranteedTicketCountMode	Manual · Poisson	0 · 1
(tutorial kind)	tutorial.steps[].kind	ForcedMove · "" (boş)	—
FoodCategory	JSON'da yok	Main · Side · Drink	0 · 1 · 2
ModificationDirection	JSON'da yok	AdditionOnly · RemovalOnly · Both	0 · 1 · 2
LayerVisibility	JSON'da yok	AlwaysVisible · VisibleByDefault · HiddenByDefault	0 · 1 · 2
TicketState	runtime-only	Active · Delivered · Cancelled	0 · 1 · 2
TipTier	runtime-only	Full · Warning · Critical	0 · 1 · 2

DIKKAT – PATIENCETYPE ORDINAL SIRASI

`PatienceType` deklarasyon sırası **Impatient, Normal, Patient**'tir. Rastgele sabır çekimi `(PatienceType)random.Next(0,3)` yaptığı için bu sıra önemlidir. Ama `DayContentGenerator`'daki **Patience Mix sırası bunun tam tersidir** (`PatienceOrder = {Patient, Normal, Impatient}`) — kasten, çünkü enum sırasıyla üretilse gün en sert biletlerle başlardı. İkisini karıştırma.

1.5 Eski dosyada alan eksikse — özet

EKSIK OLAN	RUNTIME (DAYCATALOGPARSER)	EDITÖR (DAYEDITORMODEL)
boardDistribution bloğu	Gün düşer, LogError	Tasarım default'ları ile açılır
ticketRuntime bloğu	Gün düşer, LogError	45 / 90 / 150 / 10 ile açılır
ticketGeneration bloğu	Okunmaz	Default'lar; Generate base asset'i kullanır
maxModificationCount	10'a normalize	10'a normalize
patience mix (3 alan)	Okunmaz	0/0/0 = "yazılmamış" → uniform roll
tutorial bloğu	Tutorial null	Dokunulmadan taşınır (UI yok)

Editör kasten daha hoşgörülüdür. Runtime katı, editör toleranslı — böylece eski bir gün açılıp yeniden kaydedilerek güncel şemaya taşınabilir. Editör ayrıca aralık dışı değerleri *yükleme anında* clamp'ler, yoksa dışarıda düzenlenmiş bozuk bir dosya olduğu gibi geri yazılırdı.

§2 Ticket generation algoritması

"Generate" butonunun tam akışı ve rastgele çekimlerin sırası.

```
DayContentGenerator.Generate
```

```
TicketFactory.Create
```

```
TruncatedPoisson.Sample
```


2.1 Üst seviye akış

```

Generate(catalog, baseTicketConfig, editorMeta, ticketsRequiredForDay, seed):
    random = new System.Random(seed)           // TEK rastgelelik kaynağı
    config = ResolveTicketGenerationConfig(...) // asset klonu, çekim yok
    pool = ResolveFoodPool(catalog, editorMeta) // çekim yok
    return GenerateCore(pool, config, N, random, editorMeta)

GenerateCore:
    factory = new TicketFactory(config, random)
    patiencePlan = BuildPatiencePlan(p, n, i, N) // deterministik, çekim yok
    for idx = 0 .. N-1:
        patience = patiencePlan.Count > 0
                    ? patiencePlan[idx]           // ÇEKİM YOK
                    : factory.PickRandomPatienceType() // 1 çekim
        ticket = factory.Create(pool, "Simulated Customer", patience)
        out[idx] = ToTicketEntryJson(ticket)

```

FOOD POOL ÇÖZÜMLEMESİ

`DayContentGenerator.ResolveFoodPool`

Havuz, **katalog sırası korunarak** filtrelenir — id listesi üzerinden map yapılmaz:

```
pool = catalog.Items.Where(i => i != null && allowedIds.Contains(i.Id))
```

Sonuçları: (a) katalog sırası korunur, (b) id listesindeki tekrarlar item'ı çoğaltmaz, (c) silinmiş bir asset'in id'si null entry üretmez, sadece yok sayılır. **Boş seçim = boş havuz**, "her şey" demek değil — bu durumda `Create` exception atar (editör önceden engeller).

PATIENCE PLAN

`DayContentGenerator.BuildPatiencePlan`

```

BuildPatiencePlan(patient, normal, impatient, N):
    if N <= 0 → []
    if patient<=0 && normal<=0 && impatient<=0 → [] // "yazılmamış"

    counts = [max(0,patient), max(0,normal), max(0,impatient)]
    order = [Patient, Normal, Impatient] // enum sırası DEĞİL
    plan = []
    for i in 0..2:
        for n in 0..counts[i]-1:
            if plan.Count >= N: break // fazlası KUYRUKTAN kesilir
            plan.Add(order[i])
    while plan.Count < N: plan.Add(Normal) // eksik Normal ile doldurulur
    return plan

```

Fazlalık kuyruktan kesildiği için önce Impatient, sonra Normal kaybolur. Eksik **orantısal değil, Normal ile** doldurulur — nötr tip, günü sessizce kolaylaştırmayan/zorlaştırmayan tek seçim.

2.2 Tek bir biletin üretimi — çekim sırası

`TicketFactory.Create`

Bu sıra **birebir** uygulanmalıdır; bir çekimin atlanması veya sırasının değişmesi tüm akışı kaydırır.

#	ADIM	ÇEKİM	KOŞUL
0	Patience (plan yoksa)	<code>Next(0, 3)</code>	Yalnız <code>patiencePlan</code> boşsa
1	Main seçimi	<code>NextDouble()</code>	<code>totalWeight > 0</code> ise. Değilse <code>Next(mains.Count)</code>
2	Side dahil mi	<code>NextDouble()</code>	Her zaman çekilir
3	Side hangisi	<code>Next(cand.Count)</code>	Yalnız #2 geçtiyse ve aday varsa
4	Drink dahil mi	<code>NextDouble()</code>	Her zaman çekilir
5	Drink hangisi	<code>Next(cand.Count)</code>	Yalnız #4 geçtiyse ve aday varsa
6	Modifikasyon sayısı	<code>NextDouble()</code>	Yalnız <code>maxMods > 0</code> ise
7	Hangi modifikasyonlar	<code>Next(i, count)</code>	Seçilen her modifikasyon için birer kez
8	Her modifikasyonun yönü	<code>NextDouble()</code>	Yalnız <code>Both</code> yönlü olanlar için

SIK YAPILACAK ÜÇ HATA

- 1. Side/Drink şans çekimi koşulsuzdur.** `TryAddRandomItem` önce `NextDouble()` çeker, *sonra* aday listesine bakar. Havuzda hiç içecek olmasa bile çekim yapılır ve akış kayar.
- 2. `TruncatedPoisson.Sample`, `n == 0` iken hiç çekim yapmadan 0 döner.** Yani modifikasyonu olmayan bir ana yemek akıştan hiçbir sayı tüketmez.
- 3. Yön çekimi yalnız `Both` için yapılır.** `AdditionOnly` / `RemovalOnly` yönünü kendi tanımından alır, çekim yoktur.

MAIN DISH AĞIRLIK NORMALİZASYONU

`TicketFactory.PickWeightedMain · ResolveWeight`

```

ResolveWeight(food):
    foreach entry in config.MainDishWeights:
        if entry.Food == food: return max(0, entry.Weight)
    return 1.0 // MainDishWeight.DefaultWeight

PickWeightedMain(mains):
    weights[i] = ResolveWeight(mains[i]) // double olarak biriktirilir
    totalWeight = Σ weights
    if totalWeight <= 0: return mains[random.Next(mains.Count)]

    roll = random.NextDouble() * totalWeight
    cum = 0
    for i in 0..mains.Count-1:
        cum += weights[i]
        if roll < cum: return mains[i]
    return mains[^1] // float artığına karşı

```

Yani **normalizasyon yoktur** — ağırlıklar toplanır, rulet çarkı toplam üzerinden çevrilir. Ağırlıkların 1'e veya 100'e toplanması gerekmez. `mains` listesinin sırası havuz sırasındır, o da katalog sırasındır.

Listede olmayan bir Main varsayılan 1.0 ağırlık alır. Day Editor listeyi seçili Main'lerle senkron tuttuğu için pratikte tetiklenmez, ama elle düzenlenmiş bir dosyada tetiklenir — simülatör bunu uygulamalı.

SIDE VE DRINK HAVUZU

`TicketFactory.TryAddRandomItem`

```

TryAddRandomItem(pool, category, inclusionChance, requiredItems):
    if random.NextDouble() >= inclusionChance: return // ÇEKİM HER ZAMAN
    candidates = pool.Where(i => i.Category == category)
    if candidates.Count == 0: return
    requiredItems.Add(candidates[random.Next(candidates.Count)])

```

Her ikisi de aynı `pool` 'dan (Day'in food selection'ı) çekilir, kategoriye göre filtrelenir, **uniform** seçilir. Side/drink için ağırlık sistemi yoktur.

POISSON IMPLEMENTASYONU

`Core/TruncatedPoisson.cs`

Gerçek Poisson pmf'i değil, **normalize edilmemiş terim oranı** kullanılır. $e^{-\lambda}$ çarpanı toplama bölününce sadeleştiği için hiç hesaplanmaz:

```

Terms(n, λ):
    terms[0] = 1
    for k = 1..n: terms[k] = terms[k-1] * λ / k
    return terms // uzunluk n+1

Sample(n, λ, random):
    if n == 0: return 0 // ÇEKİM YOK
    terms = Terms(n, λ)
    total = Σ terms
    roll = random.NextDouble() * total
    cum = 0
    for k = 0..n:
        cum += terms[k]
        if roll < cum: return k
    return n

Probabilities(n, λ): // editör önizlemesi, çekim yok
    terms = Terms(n, λ); return terms.map(t => t / Σterms)

```

`terms[0]` her zaman tam olarak 1 olduğu için toplam her zaman ≥ 1 — sıfıra bölme riski yoktur. $\lambda = 0$ iken `terms = [1,0,0,...]`, sonuç daima 0.

MAXMODIFICATIONCOUNT UYGULANIŞI

`TicketFactory.Create · ResolveMaxModificationCount`

```

λ = ResolveModificationCountLambda(main)
    // listede varsa entry'nin λ'sı, yoksa config.ModificationCountLambda
maxMods = min(main.AvailableModifications.Count,
    ResolveMaxModificationCount(main))
    // listede varsa entry.MaxModificationCount (normalize edilmiş),
    // yoksa DefaultMaxModificationCount = 10
count = TruncatedPoisson.Sample(maxMods, λ, random)

```

İki tavan buluşur ve küçüğü kazanır. Yemeğin kendi listesinden fazlasını istemek imkânsızdır — `MaxModificationCount` 'u 10 yapmak, 3 modifikasyonu olan bir yemekte hiçbir şey değiştirmez.

MODİFİKASYON SEÇİMİ — TEKRAR MÜMKÜN MÜ?

`TicketFactory.ChooseModifications`

Hayır. Kısmi Fisher-Yates ile *yerine koymadan* uniform alt küme seçilir:

```

ChooseModifications(available, count):
    shuffled = copy(available)
    limit    = min(count, shuffled.Count)
    for i = 0..limit-1:
        swapIndex = random.Next(i, shuffled.Count) // [i, Count)
        swap(shuffled[i], shuffled[swapIndex])
    return shuffled[0..limit]

```

Aynı modifikasyon bir bilette iki kez çıkamaz. Not: `random.Next(i, count)` .NET'te **üst sınır hariçtir** — TS portunda `i + floor(rand() * (count - i))`.

YÖN BELİRLEME

`TicketFactory.CreateModification`

```

isAddition = modConfig.AllowedDirection switch
    AdditionOnly → true // çekim yok
    RemovalOnly  → false // çekim yok
    Both         → random.NextDouble() < ModificationAdditionChance

```

2.3 Seed ve tekrarlanabilirlik

`DayEditorModel.Generate`

`DayContentGenerator.Generate`

`Generate` bir `seed: int` parametresi alır ve `new System.Random(seed)` ile tek bir akış kurar. Ancak Day Editor bunu şöyle çağırır:

```

var seed = UnityEngine.Random.Range(int.MinValue, int.MaxValue);
var result = DayContentGenerator.Generate(catalog, config, meta, N, seed);

```

Seed hiçbir yere kaydedilmez. Kullanıcıya gösterilen bir seed alanı yoktur; her tıklama farklı sonuç verir ve bu kasten böyledir. Üretilmiş bir günü seed'inden yeniden üretmek mümkün değildir — üretilen `ticketSequence` tek kayıttır.

NEEDS DECISION – SYSTEM.RANDOM TAŞINAMAZ

.NET'in `System.Random` 'ı Knuth'un subtractive generator'ının bir varyantıdır. **TypeScript'te birebir yeniden üretilemez** (implementasyonu .NET Framework ve .NET Core arasında bile değişti). Dolayısıyla simülatör, Unity'nin belirli bir seed'ile *aynı bilet dizisini* üretmez.

Önerilen yaklaşım: simülatöre **RNG'yi dışarıdan enjekte et** (`rng: () => number`) ve doğruluğu *dizi eşitliği* ile değil **dağılım eşitliği** ile doğrula — 10^5 bilet üret, ana yemek frekanslarını, modifikasyon sayısı histogramını ve side/drink oranlarını Unity'nin ürettikleriyle karşılaştır. Belirli test için kendi seedlenebilir PRNG'ni kullan (mulberry32 vb.).

Ayrıca `Next(a, b)` ve `NextDouble()` farklı sayıda bit tüketir; "aynı çekim sırası" ancak *kendi* RNG'nde anlamlıdır.

ÜRETİLEN JSON'A YAZILMAYANLAR

`DayContentGenerator.ToTicketEntryJson`

Üretim sırasında oluşan bir `Ticket` nesnesinin şu alanları **diske yazılmaz**: müşteri adı (sabit "Simulated Customer" kullanılır ve atılır), `arrivalSequence`, ve süre. Üretilen entry'de `customerNameOverride = ""` ve `timeLimitSecondsOverride = 0` olarak sabitlenir.

Ayrıca `PickRandomCustomerPortrait()` bu yolda **kasten çağrılmaz** — çağrılıysa seed akışını kaydırır ve mevcut her gün aynı seed'le farklı içerik üretirdi.

§3 Runtime board generation

Yeni bilet geldiğinde tahtaya ne düştüğünün tam akışı.

`BoardDistributor.OnOrderPlaced`

`GameManager.OnTicketAssigned`

3.1 Tetikleme

Üretim **sipariş tetiklemelidir** — zamanlayıcı ya da her-kare yoklama yoktur. Zincir tamamen senkrondur:

```

TicketSlotManager.AssignTicket(slot)
└─ state.TicketAssigned.Publish((slot, ticket))
    └─ GameManager.OnTicketAssigned
        └─ if openingAssignmentsWithoutDistribution > 0 → sayacı azalt, ATLA
            └─ else
                activeTickets = State.TicketSlots.Where(t => t != null)
                lookaheadCount = max(3, CurrentDay.TicketRuntime.UpcomingQueueSize)
                upcoming = provider.PeekUpcoming(lookaheadCount)
                boardDistributor.OnOrderPlaced(activeTickets, upcoming)
            └─ TrayManager.OnTicketAssigned(slot) // her koşulda

```

Dikkat edilecek noktalar:

- **activeTickets** **filtresi yalnızca null kontrolüdür** — `Delivered / Cancelled` durumundaki bir bilet hâlâ dizide durabilir ve buraya geçer. Durum filtresi bir alt katmanda, `SelectGuaranteedTickets` içinde uygulanır.
- **Ticket null olsa bile çağrılır.** Bilet dizisi bittiğinde slot boş kalır ve yine bir `TicketAssigned` yayınlanır; bu kasten atlanmaz — atlandığında günün son biletleri hiç garanti turu almadan kalıyordu (kayıtlı bir playtest bug'ı).
- **Lookahead tabanı 3'tür.** `UpcomingQueueSize` daha küçük olsa bile en az 3 bilet ileriye bakılır.
- `PeekUpcoming` cursor'u ilerletmez ve **örnekleri cache'ler** — aynı index için sonra `NextTicket()` çağrıldığında *tam olarak aynı* nesne döner. Bu, leak dedup'ının referans kimliğine dayanması için zorunludur.

3.2 OnOrderPlaced — okuma sırası

```

OnOrderPlaced(activeTickets, upcomingTickets):
    SpawnMissingRequiredItems(activeTickets, upcomingTickets) // ÖNCE
    LeakNoiseItems(upcomingTickets) // SONRA

```

Sıra kilitli bir tasarım kuralıdır: gerekli havuz her zaman gürültüden önce spawn eder. Board dolmaya başladığında gerekli item'ların yer bulma önceliği buradan gelir.

3.3 Missing required items hesabı

```
BoardDistributor.SpawnMissingRequiredItems · CountItemsOnBoard
```

```

SpawnMissingRequiredItems(active, upcoming):
    guaranteed = SelectGuaranteedTickets(active, upcoming)
    if guaranteed.Count == 0: return

    // 1) İhtiyaç: garantili biletlerin item'ları toplanır
    neededCounts = {}
    foreach ticket in guaranteed:
        foreach food in ticket.RequiredItems:
            mods = (food.Category == Main) ? ticket.Modifications : []
            key = RequiredItemKey(food, mods)
            neededCounts[key] += 1 // TOPLANIR

    // 2) Mevcut: tahtadaki her hücre taranır
    presentCounts = {}
    for x = 0..Width-1: for y = 0..Height-1:
        item = Board.ItemAt(x, y)
        if item != null: presentCounts[RequiredItemKey(item.Config, item.Modifications)] += 1

    // 3) Fark kadar spawn
    foreach (key, needed) in neededCounts:
        present = presentCounts[key] ?? 0
        for i = present .. needed-1:
            Board.RequestSpawn(BoardItem(key.Food, key.Modifications), random)

```

REQUIREDITEMKEY – EŞİTLİK KURALI

Core/RequiredItemKey.cs

Bir item'ı (**yemek + modifikasyon kümesi**) olarak tanımlar. Modifikasyon listesi bir HashSet imzasına çevrilir, yani **sıra bağımsızdır** ve SetEquals ile karşılaştırılır. Hash, XOR ile birleştirilir (yine sıra bağımsız).

TS karşılığı: modifikasyonları (modId, isAddition) çiftleri olarak sırala, deterministik bir string'e serialize et, Map anahtarı yap. Örn:

```
`${foodId}|${sortedMods.join(',')}`.
```

MODİFİKASYONLAR YALNIZ ANA YEMEĞE AİTTİR

TicketRequirements.KeyFor

Bir biletin modifikasyonları **yalnız Main kategorisindeki item'a** uygulanır; Side ve Drink için boş liste kullanılır. Bu kural teslimatın doğru sayılıp sayılmadığını belirlediği için kritiktir ve TicketRequirements altında tek bir yerde toplanmıştır.

Bilinen tekrar: BoardDistributor aynı kuralı iki yerde daha kendi eliyle yazar (SpawnMissingRequiredItems ve LeakNoiseItems). Kod yorumunda bilerek kayıt altına alınmış bir borç. Simülatörde tek fonksiyona indir.

BIRDEN FAZLA BİLET AYNİ ITEM'I İSTERSE

Sayılar toplanır. Üç garantili bilet de hamburger istiyorsa `neededCounts[burger] = 3` olur ve tahtada 1 tane varsa 2 tane daha spawn edilir. Ancak dikkat: modifikasyonlar anahtarın parçası olduğu için "ekstra hardallı hotdog" ile "düz hotdog" **farklı anahtarlardır** ve birbirinin yerini tutmaz.

3.4 Guaranteed ticket seçimi — pseudocode

```
BoardDistributor.SelectGuaranteedTickets
```

Bu, sistemin en karmaşık parçasıdır. `guaranteedTickets` bir **instance alanıdır** — turlar arasında yaşar (sticky).

SelectGuaranteedTickets(activeTickets, upcomingTickets):

```
// — 1. Havuz: aktif + kuyruk, varış sırasına göre —————
pool = activeTickets.Where(t => t != null && t.State == Active)
    .Concat(upcomingTickets)
    .OrderBy(t => t.ArrivalSequence)

// — 2. Budama: havuzdan çıkanlar garantiden de çıkar —————
guaranteedTickets.IntersectWith(pool)

// — 3. Aciliyet: koşulsuz eklenir, bütçeden ÖNCE —————
foreach ticket in activeTickets:
    if ticket != null
        && ticket.State == Active
        && ticket.RemainingSeconds < settings.UrgentTimeThresholdSeconds:
            guaranteedTickets.Add(ticket) // STRICT < (eşitlik dahil değil)

// — 4. Bütçe —————
budget = (mode == Poisson)
    ? TruncatedPoisson.Sample(3 - 1, GuaranteedTicketCountLambda, random) + 1
    : GuaranteedTicketCount

// — 5. Bütçe YALNIZCA aktif biletleri sayar —————
activeSet = HashSet(activeTickets.Where(t => t != null && t.State == Active))
slotsToFill = budget - guaranteedTickets.Count(t => activeSet.Contains(t))

if slotsToFill > 0:
    // 5a. ÖNCE aktifler (varış sırası korunur)
    activeCandidates = pool.Where(t => activeSet.Contains(t)
        && !guaranteedTickets.Contains(t))
    foreach picked in WeightedSampleWithoutReplacement(activeCandidates, slotsToFill):
        guaranteedTickets.Add(picked)
        slotsToFill--

    // 5b. Her aktif bilet kapsandıktan SONRA kuyruk
    if slotsToFill > 0:
        upcomingCandidates = pool.Where(t => !activeSet.Contains(t)
            && !guaranteedTickets.Contains(t))
        foreach picked in WeightedSampleWithoutReplacement(upcomingCandidates, slotsToFill):
            guaranteedTickets.Add(picked)

return guaranteedTickets.ToList()
```

MANUAL VE POISSON MODUN FARKI

MOD	BÜTÇE FORMÜLÜ	ARALIK	ÇEKİM
Manual	GuaranteedTicketCount	1-3 (ctor clamp)	Yok
Poisson	Sample(2, λ , rng) + 1	1-3	$1 \times \text{NextDouble}()$

+1 kaydırması kritiktir. Çekim `0..TicketSlotCount-1` aralığında yapılır ve sonra 1 eklenir, böylece bütçe asla 0 olamaz — "her zaman en az bir bilet tamamlanabilir olmalı" kuralı bir clamp ile değil bu kaydırmayla sağlanır. Simülatörde `Sample(2,  $\lambda$ )` yazıp `+1` 'i unutmak, olmayan bir "0 bilet" durumu üretir.

$\lambda = 10$ 'da bile sonuç $\sim 82\%$ ihtimalle 3'tür; 99% için $\lambda \approx 199$ gerekir. "Her zaman üç" istiyorsan Manual + 3 doğru araçtır.

EARLYTICKETWEIGHTDECAY – BİREBİR FORMÜL

`BoardDistributor.WeightedSampleWithoutReplacement`

```

WeightedSampleWithoutReplacement(pool, count):
    remaining = copy(pool)           // varış sırasında, asla yeniden sıralanmaz
    result    = []

    for picks = 0 .. count-1:
        if remaining.Count == 0: break

        // — HER ÇEKİMDE yeniden hesaplanır —
        for i = 0 .. remaining.Count-1:
            weights[i] = Math.Pow(EarlyTicketWeightDecay, i)
        totalWeight =  $\Sigma$  weights

        roll        = random.NextDouble() * totalWeight
        cum          = 0
        chosenIndex = remaining.Count - 1      // fallback
        for i = 0 .. remaining.Count-1:
            cum += weights[i]
            if roll < cum: chosenIndex = i; break

        result.Add(remaining[chosenIndex])
        remaining.RemoveAt(chosenIndex)       // index'ler kayar

    return result

```

Ağırlık **bilete değil, kalan listedeki sıraya** bağlıdır. `remaining` hep varış sırasında kaldığı için index 0 daima "henüz seçilmemişlerin en erkeni"dir. Uç değerler:

- `decay = 0` → $0^0 = 1$, $0^i = 0$ ($i \geq 1$) → **deterministik en-erken-önce.**
- `decay = 1` → tüm ağırlıklar 1 → **uniform.**

Not: JS'de `Math.pow(0, 0)` da `1` döner, uyumlu.

ACILİYET İLE BÜTÇE ETKİLEŞİMİ

Sıralama şöyledir ve sonucu üç maddede özetlenebilir:

1. **Acil biletler bütçeden önce eklenir** (adım 3), yani bütçe hesabı onları zaten kapsanmış sayar.
2. **Bütçe içindeyken aciliyet bütçe tüketir:** bütçe 2 ve 1 acil bilet varsa `slotsToFill = 1`.
3. **Bütçeyi aşarsa aciliyet kazanır:** bütçe 1 ve 3 acil bilet varsa `slotsToFill = -2` olur, ek seçim yapılmaz, ama üç acil bilet de garantilidir. Aciliyet her zaman kazanır.

Yalnız **aktif** biletler acil olabilir: kuyruktaki bir biletin `RemainingSeconds` 'ı slota girene kadar `TimeLimitSeconds` 'ta donmuştur.

STICKY SEÇİM – ATLANAMAZ

`guaranteedTickets` her turda sıfırdan hesaplanmaz; budanır ve üstüne eklenir. Her `OnOrderPlaced` 'de piyango yeniden çevrilseydi, hiçbir şey spawn edilmiş item'ı geri almadığı için "en az bir bilet" garantisi tekrarlı çekimlerle sessizce "eninde sonunda her bilet"e kayardı. Bir bilet garanti kümesinden yalnızca havuzdan tamamen çıkınca (teslim/iptal) düşer.

Simülatörde bu, `BoardDistributor` 'ın **durumlu (stateful)** olması gerektiği anlamına gelir: `guaranteedTickets` ve `leakedTickets` kümeleri gün boyunca yaşar ve **gün/retry değişiminde sıfırdan kurulur** (`GameManager.RefreshDayTicketSequenceProvider` distributor'ı yeniden inşa eder).

§4 Noise / leak algoritması

Tahtaya sızan "işe yaramaz" item'lar — zorluğun asıl kaynağı.

```
BoardDistributor.LeakNoiseItems
```

LeakNoiseItems(upcomingTickets):

```
// 1. Budama: kuyruktan çıkmış biletler dedup setinden düşer
leakedTickets.IntersectWith(upcomingTickets)

// 2. Adaylar: LeakDepth kadar ÖNDEN, henüz sızmamış olanlar
candidates = upcomingTickets.Take(LeakDepth)
                                .Where(t => !leakedTickets.Contains(t))
if candidates.Count == 0: return           // ÇEKİM YOK – erken çıkış

// 3. Kaç tane sızacak
leakCount = min( TruncatedPoisson.Sample(MaxLeakCount, NoiseLeakCountLambda, random),
                candidates.Count )

// 4. Sızdırma döngüsü
for i = 0 .. leakCount-1:
    index          = random.Next(candidates.Count)      // ① ticket seçimi
    sourceTicket = candidates[index]
    candidates.RemoveAt(index)                          // yerine koymadan

    food = sourceTicket.RequiredItems[random.Next(sourceTicket.RequiredItems.Count)] // ② item seçimi
    mods = (food.Category == Main) ? sourceTicket.Modifications : []

    Board.RequestSpawn(BoardItem(food, mods), random) // ③ hücre seçimi
    leakedTickets.Add(sourceTicket)
```

Soru-cevap

SORU	CEVAP
MaxLeakCount nerede uygulanır?	<code>TruncatedPoisson.Sample</code> 'ın <code>n</code> parametresi olarak — yani dağılımın kesme noktasıdır , sonradan uygulanan bir clamp değil. Bu yüzden editördeki olasılık önizlemesi yalnız <code>(MaxLeakCount, λ)</code> 'ya bağlıdır ve kuyrukta kaç bilet olduğundan bağımsızdır.
Sonra ikinci bir sınır var mı?	Evet: <code>min(sample, candidates.Count)</code> . Örn. <code>MaxLeakCount=5</code> ama <code>LeakDepth</code> içinde 2 sızmamış bilet varsa en fazla 2 sızar.
LeakDepth ne belirler?	Yalnız hangi biletlerin aday olduğunu — kaç item sızacağını değil. <code>Take(LeakDepth)</code> , kuyruğun önünden (index 0 = en yakın gelecek bilet) o kadarını alır.
Seçim ticket bazlı mı item bazlı mı?	Önce ticket, sonra item. Uniform bir bilet seçilir (①), sonra o biletin <code>RequiredItems</code> listesinden uniform bir item seçilir (②). Yani çok item'lı bir bilet, her item'ı için ayrı şans <i>almaz</i> ; bilet başına tek item sızar.
Aynı item bir spawn'da iki kez gelebilir mi?	Evet. Bilet seçimi yerine koymadan yapılır (aynı bilet iki kez seçilemez), ama iki <i>farklı</i> bilet aynı yemeği içerebilir ve ikisi de o yemeği sızdırabilir.
Aynı bilet birden çok kez sızdırır mı?	Hayır. Ne aynı spawn içinde (<code>RemoveAt</code>), ne de kuyrukta beklediği sürece (<code>leakedTickets</code>). Bilet kuyruktan çıkıp slota girince set budanır; onun yerine gelen yeni bilet hemen uygun hale gelir.
Aktif biletin de ihtiyacı olan bir item noise olarak seçilebilir mi?	Evet. Kaynak yalnız kuyruktur, ama sızan yemek aktif bir biletin istediğiyle çıkışabilir. Üstelik bu item bir sonraki turda <code>presentCounts</code> 'a sayılır ve o biletin gerekli-havuz spawn'ını azaltır. Bu <i>engellenmiş bir durum değildir</i> .
Board doluysa?	<code>RequestSpawn</code> boş hücre bulamazsa item'ı <code>pendingSpawns</code> kuyruğuna alır ve <code>false</code> döner. Kimse dönüş değerini kontrol etmez. Bir hücre boşaldığında <code>RemoveItem</code> o hücreyi kuyruktan doldurur. Bkz. §5.3.

§5 Board state

Izgaranın boyutu, hücre modeli, yerleştirme ve kaldırma.

[Core/BoardGrid.cs](#)
[Data/GameConfig.cs](#)

5.1 Boyut ve model

Boyut `GameConfig` `ScriptableObject`'inden gelir — **Day JSON'da değildir**, yani tüm günler aynı tahtayı paylaşır:

```
[SerializeField] private int boardWidth = 6;
[SerializeField] private int boardHeight = 5;
// BoardGrid ctor: cells = new BoardItem[Width, Height]
```

Bunlar serialize edilmiş alanlardır; `.asset` dosyasındaki değer kodda yazan default'u ezebilir. **Simülatörde board boyutunu parametrik tut**, 6×5'i sabitleme.

ÜYE	TIP	ANLAMI
cells	BoardItem[Width,Height]	Hücre başına 0 veya 1 item. Yığın yok.
pendingSpawns	Queue<BoardItem>	Yer bulamayan item'lar (FIFO).
OccupiedCellCount	int	Dolu hücre sayısı.
CellCount	int	Width * Height
IsFull	bool	OccupiedCellCount >= CellCount
CellChanged	EventBus<(int,int)>	Yalnız koordinat taşır; abone <code>ItemAt</code> ile okur.

Bir `BoardItem` (`FoodItemConfig Config`, `IReadOnlyList<Modification> Modifications`) taşır — yani modifikasyonlar item'ın üzerindedir, hücrenin değil.

5.2 Hücre seçimi — random mı deterministik mi?

`BoardGrid.RequestSpawn · TryGetEmptyCell`

Çağırana bağlıdır. `RequestSpawn` 'un `random` parametresi opsiyoneldir ve iki farklı davranış üretir:

```
TryGetEmptyCell(random, out x, out y):
    if random == null:
        return TryGetFirstEmptyCell(out x, out y) // DETERMİNİSTİK

    emptyCells = []
    for scanY = 0..Height-1: // dış döngü Y
        for scanX = 0..Width-1: // iç döngü X
            if cells[scanX, scanY] == null: emptyCells.Add((scanX, scanY))
    if emptyCells.Count == 0: return false
    (x, y) = emptyCells[random.Next(emptyCells.Count)] // UNIFORM
```

`TryGetFirstEmptyCell` aynı tarama sırasını (Y dış, X iç) kullanır ve ilk boşu döner.

ÇAĞIRAN	RANDOM VERİLİR MI?	SONUÇ
BoardDistributor.SpawnMissingRequiredItems	evet	Uniform rastgele hücre
BoardDistributor.LeakNoiseItems	evet	Uniform rastgele hücre
TrayManager.ScatterBackToBoard	evet	Uniform rastgele hücre
DayBoardTimelinePlayer.ApplyForStep	hayır	İlk boş hücre — kasten deterministik
BoardItemDragHandler (geçersiz bırakma)	hayır	İlk boş hücre
WorldTrayView (tepsiden geri)	hayır	İlk boş hücre

5.3 Kaldırma ve backfill

`BoardGrid.RemoveItem`

```

RemoveItem(x, y):
    if !IsInBounds(x,y): return null
    removed = cells[x,y]
    if removed == null: return null

    cells[x,y] = null
    OccupiedCellCount--

    if pendingSpawns.Count > 0:
        TryPlaceItem(pendingSpawns.Dequeue(), x, y)    // AYNI hücreye backfill
                                                         // CellChanged'i kendi yayınlar
    else:
        CellChanged.Publish((x,y))

    return removed

```

Backfill anında ve aynı hücreye olur. Bu, oyuncunun bir item'ı aldığı anda o hücrede yeni bir item belirebileceği anlamına gelir. `Clear()` bu yüzden önce `pendingSpawns.Clear()` yapar — sonra yapsaydı döngü sırasında hücreler yeniden dolar ve tahta hiç boşalmazdı.

5.4 Oyuncu item kullandığında

`BoardItemDragHandler.DetachFromBoard` `TrayManager.TryAddItem`

Kaldırma, sürükleme başında değil **tepsi kabul ettiği anda** olur ve sıralaması kasten böyledir:


```
WorldTrayView → TrayManager.TryAddItem(slotIndex, item, dragHandler.DetachFromBoard)
```

```
TryAddItem(slotIndex, item, onAccepted):
    ticket = state.TicketSlots[slotIndex]
    if ticket == null || ticket.State != Active: return false
    if slot.IsFull(ticket): return false

    onAccepted?.Invoke() // ← board.RemoveItem BURADA çalışır
    slot.Add(item)

    if slot.IsFull(ticket): // items.Count >= ticket.RequiredItems.Count
        if slot.Matches(ticket):
            slot.Clear() // ÖNCE temizle
            deliverTicket(slotIndex) // SONRA teslim (senkron cascade)
        else:
            loseLife(slotIndex)
            ScatterBackToBoard(slotIndex, slot)
            slot.Clear()

    return true
```

`onAccepted` 'ın batch check'ten *önce* çalışması zorunludur: teslimat senkron olarak `OnOrderPlaced` 'e kadar iner ve o sırada item hâlâ eski hücresinde durursa, başka bir aktif biletle paylaşılan bir kombinasyon "zaten tahtada var" diye okunur ve hiç yenilenmez.

5.5 Kullanılmayan item'lar ne kadar kalır?

Süresiz. Kodda item'lara bağlı hiçbir zamanlayıcı, ömür veya çürüme yoktur. Bir item tahtadan yalnız şu yollarla çıkar:

- Oyuncu alıp bir tepsiye koyar (`DetachFromBoard`).
- `BoardGrid.Clear()` — gün sıfırlaması.
- Noise Clear powerup'ı (`NoiseClearRunner`) — aktif biletlerin hâlâ ihtiyaç duyduğu sayının üzerindeki kopyaları siler.

Yani tahta yalnızca birikir. Bir balancing simülatörünün ölçmesi gereken asıl metrik budur: **doluluk oranının zaman içindeki eğrisi** ve `pendingSpawns` 'ın büyümesi.

§6 Ticket lifecycle

Oluşturmadan teslimat veya timeout'a kadar durum akışı.

`TicketSlotManager` `Core/Ticket.cs`

6.1 Slot sayısı

```
public const int TicketSlotCount = 3;           // Core/GameState.cs
public Ticket[] TicketSlots { get; } = new Ticket[TicketSlotCount];
```

Bir `const` 'tur, config'ten gelmez — kilitli bir tasarım kuralıdır. Poisson bütçe formülü (`Sample(TicketSlotCount - 1, ...)`) ve tutorial tray doğrulaması bu sabiti okur.

6.2 Durumlar

```
enum TicketState { Active, Delivered, Cancelled }
// Ticket ctor: State = Active, RemainingSeconds = timeLimitSeconds
```

Geçişler tek yönlüdür ve yalnız iki yerden yazılır: `DeliverTicket` ($\rightarrow$ Delivered) ve `CancelTicket` ($\rightarrow$ Cancelled). Geri dönüş yoktur.

6.3 Teslimat zinciri — tamamen senkron

```
TrayManager.TryAddItem — tepsi doldu ve eşleşti
├ slot.Clear()
└ deliverTicket(slotIndex) → TicketSlotManager.DeliverTicket
    ├── ticket.State = Delivered
    ├── state.TicketDelivered.Publish((slotIndex, ticket))
    │   └── GameManager: ekonomi kaydı, DayLifecycleManager.RecordDelivery
    └ AssignTicket(slotIndex)
        ├── if IsDayComplete: return
        ├── ticket = nextTicketProvider() // null olabilir
        ├── state.TicketSlots[slotIndex] = ticket
        ├── state.TicketAssigned.Publish((slotIndex, ticket))
        │   └── GameManager.OnTicketAssigned
        │       ├── boardDistributor.OnOrderPlaced(...) // §3
        │       └── TrayManager.OnTicketAssigned(slotIndex)
        └ if ticket == null && AllSlotsEmpty():
            IsDayComplete = true
            state.DayCompleted.Publish(TicketsDeliveredToday)
```

Sıra: yeni bilet $\rightarrow$ sonra board generation. Board üretimi, yeni bilet slota *yerleştikten sonra* tetiklenir, yani `OnOrderPlaced` yeni bileti aktif listede görür.

6.4 Timeout

```
TicketSlotManager.Tick
```

```

Tick(deltaSeconds):
    if IsDayComplete: return
    for i = 0..2:
        ticket = state.TicketSlots[i]
        if ticket == null || ticket.State != Active: continue

        ticket.RemainingSeconds = max(0, ticket.RemainingSeconds - deltaSeconds)
        if ticket.RemainingSeconds <= 0:
            loseLife(i) // ÖNCE can
            if state.IsAwaitingContinue:
                deferredTimeouts[i] = true // son can → iptal ERTELENİR
                continue
            CancelTicket(i)

```

Evet, timeout board generation tetikler. CancelTicket → TicketCancelled.Publish
 → AssignTicket → TicketAssigned → OnOrderPlaced. Teslimatla aynı zincir.

Tek istisna: son can gittiye iptal ertelenir (deferredTimeouts), böylece Game Over popup'ının arkasında yeni item'lar patlamaz. Ertelenen iptaller Continue sonrası ilk canlı karede ResolveDeferredTimeouts ile çalışır ve o zaman board turunu tetikler.

TICK NE ZAMAN ÇALIŞMAZ

GameManager.Update

```

Update():
    if State.IsAwaitingContinue: return // Game Over popup açık
    if State.IsPaused: return // ayarlar menüsü / powerup shop
    if Tutorial != null && Tutorial.IsArmed: return // tutorial adımı

    TicketSlotManager.ResolveDeferredTimeouts()
    TicketSlotManager.Tick(Time.deltaTime)
    NotifyTutorialOfTicketPatience()

```

Üç ayrı bayrak, üç ayrı sebep — kasten birleştirilmemiş. Simülatör bir "wall clock" yerine **sabit adımlı bir tick** kullanılmalı ve bu üç durağı modelleyip modellememeye karar vermeli (balancing için genelde gereksiz).

6.5 Day completion — tam koşul

TicketSlotManager.AssignTicket

```

if ticket == null && AllSlotsEmpty():
    IsDayComplete = true
    state.DayCompleted.Publish(state.TicketsDeliveredToday)

```

Yani gün, **bilet dizisi tükendiğinde ve üç slotun üçü de boşaldığında** biter. Bu tetikleyici olan çözüm bir teslimat da olabilir, bir iptal de.

GÜN BITİŞİ TESLİMAT SAYISINA BAĞLI DEĞİL

Bu kayıtlı bir bug düzeltmesi. `nextTicketProvider()` slot başına bir kez çağrılır (gün başında 3, sonra her çözümde 1), yani dizi her zaman "N teslimat" hedefine ulaşılmadan birkaç çözüm önce tükenir – ve timeout olan bilet hiç teslim sayılmaz. Bitişi teslimat sayısına bağlamak günü **tamamlanamaz** yapıyordu.

Sonuç: `ticketsRequiredForDay` bir *bitiş koşulu değildir*, yalnızca dizinin uzunluğudur (ve validator bunu zorunlu tutar). Oyuncu bir bileti timeout'a düşürse bile gün, dizi tükenip slotlar boşalınca biter – sadece daha az teslimatla.

6.6 Bilet örnekleri nereden gelir

`DayTicketSequenceProvider`

`TicketEntryFactory.Create`

```
NextTicket():
    index = cursor; cursor++
    if peekCache.has(index): return peekCache.pop(index)    // AYNI nesne
    return TicketEntryFactory.Create(sequence[index], ticketRuntime, factory, index)

PeekUpcoming(count):                                     // cursor'u İLERLETMEZ
    for index = cursor .. min(cursor+count, len)-1:
        if !peekCache.has(index):
            peekCache[index] = TicketEntryFactory.Create(sequence[index], ..., index)
        result.Add(peekCache[index])
```

`arrivalSequence` = **dizideki index'tir** – artan bir sayaç değil. Yani varış sırası authoring sırasıyla birebir aynıdır ve ``OrderBy(ArrivalSequence)`` deterministiktir.

`TicketEntryFactory.Create` iki rastgele çekim yapar (isim ve portre) ve bunlar **seedlenmemiş** factory'den gelir; balancing açısından anlamsızdır, simülâtörde atlanabilir.

§7 Start board

Açılış tahtası nasıl yüklenir ve neden normal dağıtımı bastırır.

`GameManager.ApplyDayStartBoardPreSeed`

`DayBoardTimelinePlayer`

7.1 Yükleme

```

ApplyDayStartBoardPreSeed():
  if CurrentDay == null: return
  DayBoardTimelinePlayer.ApplyForStep(State.Board, CurrentDay.BoardTimeline, -1)

  openingAssignmentsWithoutDistribution =
    DayBoardTimelinePlayer.HasEntriesForStep(CurrentDay.BoardTimeline, -1)
      ? GameState.TicketSlotCount // 3
      : 0
  ArmTutorial()

```

Bu, tahta temizlendikten *sonra* ve ilk bilet ataması yapılmadan *önce* çalışır. Dört gün-başlangıcı yolunun (ilk yükleme, iki retry, sonraki güne geçiş) hepsi buradan geçer.

Sayaç koşulsuz atanır, 0 dahil — açılış tahtası olmayan bir gün, önceki günün bıraktığı sayacı temizlemek zorundadır.

7.2 Yerleştirme kuralı

`DayBoardTimelinePlayer.ApplyForStep`

```

ApplyForStep(board, timeline, stepIndex):
  foreach entry in timeline:
    if !PlaysOnStep(entry, stepIndex): continue
    item = BoardItem(entry.Item, entry.Modifications)
    if entry.UseExactCell && board.TryPlaceItem(item, entry.X, entry.Y): continue
    board.RequestSpawn(item) // random YOK → ilk boş hücre

PlaysOnStep(entry, stepIndex):
  entry != null && entry.TriggerStepIndex == stepIndex && entry.Item != null

```

Üç davranış:

- `useExactCell` açık ve hücre boşsa → oraya konur.
- `useExactCell` açık ama hücre **doluysa** (aynı hücreye iki entry) → sessizce ilk boş hücreye düşer.
- `useExactCell` kapalı → ilk boş hücreye.

Hiç randomness yoktur — kasten. Aynı authored gün her çalıştırmada aynı açılış tahtasını verir. Timeline listesi sırayla işlenir, yani JSON'daki dizi sırası yerleşimi etkiler.

7.3 Normal generation neden bastırılır

Açılış tahtası yazan bir gün, oyuncunun gördüğü tahtanın **tam olarak tasarımcının çizdiği şey** olmasını ister. Bastırılmasaydı üç biletlik gerekli-havuz spawn'ı üstüne binerdi.

Bastırma, `OnTicketAssigned` 'da **çağırıcıyı tamamen atlayarak** yapılır, distribütör'ü çalıştırıp sonucu atarak değil. Bu fark önemlidir: `OnOrderPlaced` bir bileti günün geri kalanı için "garantili" yapan şeydir (sticky seçim), dolayısıyla bastırılmış bir tur açılış biletlerinin piyango sırasını *tüketmemelidir*. Onlar ilk gerçek turda piyangoya girerler.

7.4 İlk gerçek spawn ne zaman?

Sayaç 3'ten başlar ve açılış `FillEmptySlots / ResetSlotsForNewDay` üç `TicketAssigned` yayınlar, üçü de yutulur. **İlk dağıtım turu, 4. atamada** — yani ilk bilet çözüldüğünde (teslim veya iptal) çalışır.

Açılış tahtası *olmayan* bir günde sayaç 0'dır ve üç açılış ataması da normal dağıtım yapar.

Sayaç **atama sayar, bilet değil**: dizisi üçten kısa bir gün de sıfıra iner, çünkü tükenmiş dizi null atama yayınlar ve o da bir açılış olayıdır.

7.5 Validation'daki "first N"

`DayValidator.ValidateDayStartBoard`

```
ticketsOnScreenAtOpen = Math.Min(GameState.TicketSlotCount, day.TicketSequence.Count)
// = min(3, bilet sayısı)
```

Yani N, ekranda açılışta olacak bilet sayısıdır: normalde 3, ama 2 biletlik bir günde 2.

Kural: **bu N bileten en az biri** açılış tahtasından tamamlanabilmelidir. "Hepsi" değil — bir tamamlanabilir bilet bir hamledir, hamle bir slot boşaltır, boşalan slot yeni bilet getirir ve bastırılan dağıtım oradan devam eder.

UNKNOWN - TRIGGERSTEPINDEX ≠ -1

`BoardSpawnEntryJson.triggerStepIndex` şemada herhangi bir `int` kabul eder ve `ApplyForStep` parametrik bir `stepIndex` alır. Ancak kodun tamamında `ApplyForStep` **yalnızca -1 ile çağrılır** (`ApplyDayStartBoardPreSeed`). Day Editor da yalnız -1 yazar.

Sonuç: -1 dışındaki bir `triggerStepIndex` hiçbir zaman tetiklenmez — item sessizce hiç spawn edilmez. Bu bir kalıntı mı yoksa gelecekteki bir özellik için mi bırakıldı, koddan anlaşılmıyor. **NEEDS DECISION:** simülatörde ya yalnız -1 'i destekle, ya da editörde başka değer yazılmasını engelle.

§8 Difficulty-relevant runtime rules

8.1 Süre çözümlemesi

`ResolvedTicketEntry.TimeLimitSecondsWith`

```
TimeLimitSecondsWith(ticketRuntime):
  if TimeLimitSecondsOverride > 0: return TimeLimitSecondsOverride
  return ticketRuntime == null ? 0 : ticketRuntime.TimeLimitSecondsFor(PatienceType)
```

Zincir **iki basamaklıdır**, üçüncü bir fallback yoktur:

1. **Override** — yalnız `> 0` ise. `0` "kapalı" demektir, "sıfır saniye" değil. Negatif de kapalıdır.
2. **Günün patience limiti** — `runtime.ticketRuntime` 'dan, oynanan günün kendi değeri.

`ticketRuntime == null` hâli yalnız authoring tarafındadır (validator için kurulan `DayDefinition`) ve `0` döner; oynanan bir günde bu mümkün değildir çünkü parser bloksuz günü reddeder.

Bu kural **tek bir yerde** durur ve iki yer okur: oyuncuya verilen süre (`TicketEntryFactory.Create`) ve yıldız puanının paydası (`DayDefinition.TotalTicketSeconds`) — ikisinin ayrışmaması için.

8.2 Timer ne zaman başlar

Üç ayrı an vardır ve karıştırılmamalıdır:

AN	NE OLUR	REMAININGSECONDS
İnşa	<code>Ticket</code> ctor (Peek veya Next ile)	<code>= TimeLimitSeconds</code>
Slota atama	<code>AssignTicket</code>	değişmez
İlk Tick	<code>TicketSlotManager.Tick</code>	azalmaya başlar

Sayaç yalnız `Tick` içinde ve yalnız `State == Active` olan slot biletleri için azalır.

`PeekUpcoming` ile önceden inşa edilmiş kuyruk biletleri tam süreyle donmuş bekler — bu yüzden bir kuyruk bileti asla "acil" olamaz.

Pratikte timer, bilet slota girdikten sonraki ilk `Update` karesinde başlar (üç durak da açık değilse).

8.3 Urgent threshold hangi değere bakar

```
ticket.RemainingSeconds < settings.UrgentTimeThresholdSeconds
```

Ham saniye, oran değil. Ve **strict <** — tam eşitlik acil sayılmaz. Yalnız `activeTickets` üzerinde, yalnız `State == Active` için değerlendirilir.

Bu, bahşiş kademeleriyle **aynı şey değildir**: kademeler oranla (`Remaining / TimeLimit`) çalışır, aciliyet mutlak saniyeyle. Bir Patient bilet (150 sn) 10 sn kala hem "acil"dir hem de bahşiş olarak zaten Critical'dır; bir Impatient bilet (45 sn) 10 sn kala aciliyete girer ama oranı 0.22'dir, o da Critical. Yine de eşikler bağımsızdır ve ayrı ayrı ayarlanır.

8.4 Tip / payout formülü

`EconomySystem/EconomyCalculator.cs`

```
CalculatePayout(ticket):
    orderValue = Σ item.BasePrice for item in ticket.RequiredItems // null atlanır

    ratio = ticket.TimeLimitSeconds <= 0
            ? 0
            : clamp01(ticket.RemainingSeconds / ticket.TimeLimitSeconds)

    tier = ratio <= config.CriticalRatio ? Critical
          : ratio <= config.WarningRatio ? Warning
          : Full

    rate = tier == Critical ? TipRateCritical
           : tier == Warning ? TipRateWarning
           : TipRateFull

    Tip = orderValue * rate
    Total = orderValue + Tip
```

KALAN ORAN	BAR RENGİ	KADEME	BAHŞIŞ ORANI
> WarningRatio (0.666)	yeşil	Full	TipRateFull
> CriticalRatio, <= WarningRatio	turuncu	Warning	TipRateWarning
<= CriticalRatio (0.333)	kırmızı	Critical	TipRateCritical

- **Order Value hiçbir zaman ölçeklenmez** — başarılı teslimat onu tam öder. Yalnız bahşiş değişir.
- **Item sayısı formüle girmez.** Büyük sipariş daha çok eder çünkü item'ları pahalıdır, ve daha düşük kademeye düşer çünkü toplaması gerçekten uzun sürer.

- **Patience formüle girmez** — yalnız verdiği süre üzerinden dolaylı etkisi vardır. Oranlar biletin kendi limitine göre olduğu için kademeler her patience tipi için otomatik ölçeklenir.
- **Critical önce test edilir ve iki sınır da $\leq$** — barın boyadığı renkle birebir aynı, eşik üstünde duran bir oran farklı kovaya düşemez.
- Eşikler ve üç oran **EconomyConfig** 'te durur (görsel config'te değil), çünkü parayı belirlerler.

Değerler `Assets/Data/EconomyConfig.asset` 'ten okunur; simülatöre parametre olarak geçilmeli.

§9 Validation kuralları

Koddan çıkarılmış tam liste. UI dökümanında olmayanlar işaretlidir.

`Systems/DaySystem/DayValidator.cs`

9.1 Errors — Save'i kapatır

#	KAYNAK METOT	KOŞUL	UI DOK.
E1	Validate	<code>TicketSequence.Count != TicketsRequiredForDay</code>	var
E2	Validate	Bir biletin Main/Side/Drink item'ı <code>allowedFoods</code> içinde değil. Bilet başına <i>ve item başına</i> ayrı hata üretir.	var
E3	ValidateDayStartBoard	Açılış tahtası var, ama ilk <code>min(3, N)</code> biletin hiçbiri ondan tamamlanamıyor.	var
E4	ValidateTicketRuntime	Üç patience süresinden herhangi biri <code><= 0</code> .	var
E5	ValidateTutorial	<code>TargetTraySlotIndex</code> 0..2 aralığı dışında.	YOK
E6	ValidateTutorial	Adımın <code>(SourceX, SourceY)</code> hücrelerine <code>triggerStepIndex == -1</code> ve <code>useExactCell == true</code> olan bir boardTimeline entry'si yok.	YOK
E7	ValidateTutorial	İki tutorial adımı aynı kaynak hücreyi kullanıyor (ilki item'ı alır, ikincisi tamamlanamaz).	YOK
E8	DayEditorModel.Validate	Config asset'leri atanmamış → <code>"Not configured yet"</code> . Validator'ın kendisi değil, editör kısa devresi.	var

9.2 Warnings – Save'i kapatmaz

#	KAYNAK	METOT	KOŞUL	UI DOK.
W1	ValidateTicketRuntime		!(Impatient < Normal < Patient) , ve Impatient>0 && Patient>0 iken.	var
W2	ValidateBoardDistribution		LeakDepth > UpcomingQueueSize	var

UI DÖKÜMANINDA OLMAYAN ÜÇ KURAL

E5, E6, E7 – tutorial doğrulamaları. El kitabında "tutorial'ın editör arayüzü yok" denmişti ama bu üç kuralın Save'i kapattığı yazılmamıştı. Pratik sonucu: `day_00` 'ın tutorial bloğunu elle bozarsan, gün **editörde kaydedilemez hale gelir** ve hata mesajı tutorial'dan bahseder – arayüzü olmayan bir şey için.

9.3 Validator'ın göremedikleri

Bu, simülatör için önemli bir sınırdır. `DayValidator` **çözümlenmiş** nesneleri inceler ve o nesnelerin ctor'ları değerleri zaten clamp'lemiştir. Dolayısıyla şu alanlar için "parser'ın reddettiğini reddet" şeklinde bir kural buraya yazılamaz:

- `UpcomingQueueSize` – `Max(1, x)` ile clamp'lenmiş
- `GuaranteedTicketCount` – `Clamp(1,3)`
- `LeakDepth` , `MaxLeakCount` – `Clamp(1,10)`

Bunların savunması başka yerdedir: **Day Editor yükleme anında clamp'ler**, böylece elle düzenlenmiş bozuk bir dosya olduğu gibi geri yazılamaz.

Süre limitleri istisnadır ve E4'ün var olma sebebidir: ctor `Max(0f, x)` kullanır, yani `0` geçer, validator görebilir.

9.4 Runtime yükleme reddi – ayrı bir katman

Bunlar *validator kuralı değildir*, elle düzenlenmiş bir dosya validator'a hiç uğramaz ve doğrudan bunlara çarpar. `DayCatalogParser`

KOŞUL	SONUÇ
Geçersiz JSON	Gün düşer, LogError
<code>runtime</code> bloğu yok	Gün düşer
Bilinmeyen <code>mainItemId</code> / <code>sideItemId</code> / <code>drinkItemId</code> / <code>itemId</code> / <code>modificationId</code>	Gün düşer
Parse edilemeyen <code>patienceType</code>	Gün düşer
<code>boardDistribution</code> yok veya <code>guaranteedTicketCountMode</code> boş	Gün düşer
Parse edilemeyen <code>guaranteedTicketCountMode</code>	Gün düşer
<code>guaranteedTicketCount < 1</code> <code>leakDepth < 1</code> <code>maxLeakCount < 1</code>	Gün düşer
Üç patience süresinden biri <code><= 0</code>	Gün düşer
<code>upcomingQueueSize < 1</code>	Gün düşer
Yinelenen <code>dayIndex</code>	Gün düşmez , sadece LogError
<code>tutorial.steps[].kind</code> ≠ "ForcedMove" ve ≠ ""	Gün düşmez , tutorial null olur

Simülatörün bu katmanı da uygulaması önerilir — harici editör, elle düzenlemenin mümkün olduğu bir dünyada validator'dan daha çok bunu taklit etmelidir.

§10 External simulator contract

Unity'den bağımsız TypeScript simülatörü için minimum saf fonksiyon kümesi.

10.1 Ortak modeller

```

type FoodId = string; type ModId = string;

interface FoodItem {
  id: FoodId; category: 'Main'|'Side'|'Drink';
  basePrice: number;
  availableModifications: ModId[];      // yalnız Main için anlamlı
}
interface ModificationDef {
  id: ModId; allowedDirection: 'AdditionOnly'|'RemovalOnly'|'Both';
}
interface Modification { modId: ModId; isAddition: boolean; }

// RequiredItemKey karşılığı – sıra bağımsız olmalı
type ItemKey = string;
function itemKey(foodId: FoodId, mods: Modification[]): ItemKey {
  const sig = mods.map(m => `${m.modId}:${m.isAddition ? 1 : 0}`).sort().join(',');
  return `${foodId}|${sig}`;
}

interface BoardItem { foodId: FoodId; mods: Modification[]; }

interface SimTicket {
  arrivalSequence: number;           // = dizideki index
  mainItemId: FoodId;
  sideItemId?: FoodId; drinkItemId?: FoodId;
  modifications: Modification[];     // yalnız main'e uygulanır
  patienceType: 'Impatient'|'Normal'|'Patient';
  timeLimitSeconds: number;          // §8.1 ile çözülmüş
  remainingSeconds: number;
  state: 'Active'|'Delivered'|'Cancelled';
}

type Rng = () => number;             // [0,1) – NextDouble karşılığı
function nextInt(rng: Rng, minIncl: number, maxExcl: number): number {
  return minIncl + Math.floor(rng() * (maxExcl - minIncl));
}

```

```
generateTicketSequence(day: DayEditorMeta, ticketsRequiredForDay: number, pool: FoodItem[],
mods: Map<ModId, ModificationDef>, rng: Rng): TicketEntryJson[]
```

Unity karşılığı: DayContentGenerator.Generate → GenerateCore → TicketFactory.Create

BİREBİR EŞLEŞMESİ GEREKENLER

- **Çekim sırası** §2.2 tablosundaki gibi, adım adım. Özellikle: side/drink şans çekimi aday yokken de yapılır; `maxMods == 0` iken Poisson hiç çekim yapmaz; yön çekimi yalnız `Both` için.
- **Patience plan** §2.1'deki gibi — sıra `[Patient, Normal, Impatient]`, fazlalık kuyruktan kesilir, eksik `Normal` ile doldurulur, 0/0/0 = uniform roll.
- **Ağırlık** normalize edilmez; listede olmayan Main `1.0` alır; `total <= 0` ise uniform.
- **maxMods** = `min(availableModifications.length, normalizeMax(entry.maxModificationCount))`.
- **Modifikasyon seçimi** kısmi Fisher-Yates, yerine koymadan.
- Çıktıda `customerNameOverride = ""`, `timeLimitSecondsOverride = 0`.

EŞLEŞMESİ GEREKMEYENLER

- Müşteri adı ve portresi (üretim yolunda çekilmez / atılır).
- Sayı-sayı aynı dizi — bkz. §2.3 RNG uyarısı. Dağılım eşitliği hedefle.

```
getMissingItems(tickets: SimTicket[], board: BoardState): Map<ItemKey, number>
```

Unity karşılığı: BoardDistributor.SpawnMissingRequiredItems 'ın ilk iki adımı + CountItemsOnBoard

SÖZLEŞME

- **Girdi tickets = garantili biletler**, aktif biletler değil. Bu fonksiyon seçim yapmaz, seçilmişleri alır.
- Her biletin her RequiredItems girdisi için needed[key] += 1 — **toplanır**.
- Modifikasyonlar **yalnız Main** için anahtara girer; Side/Drink boş liste ile.
- Tahta sayımı tüm hücreleri tarar (sıra önemsiz, sonuç bir sayım).
- Çıktı: $\max(0, \text{needed} - \text{present})$. Sıfır olanlar çıktıdan düşürülebilir.

UNKNOWN — SPAWN SIRASI BELIRSİZ

Unity tarafında foreach (var (key, needed) in neededCounts) bir Dictionary üzerinde döner. **.NET'te Dictionary iterasyon sırası spesifikasyonla garanti edilmez**. Spawn edilen *çokluk kümesi* deterministiktir, ama **hangi item'ın hangi hücreye düştüğü** sıraya bağlıdır (her RequestSpawn ayrı bir hücre çekimi yapar).

NEEDS DECISION: Simülatörde anahtarları deterministik sırala (örn. itemKey string'ine göre) ve bunun Unity'yle *hücre düzeyinde* eşleşmeyeceğini kabul et. Balancing metrikleri (kaç item, hangi item) etkilenmez; yalnız görsel yerleşim etkilenir.

```
selectGuaranteedTickets(state: DistributorState, active: SimTicket[], upcoming: SimTicket[],
settings: BoardDistributionSettings, rng: Rng): SimTicket[]
```

Unity karşılığı: `BoardDistributor.SelectGuaranteedTickets + WeightedSampleWithoutReplacement`

SAF DEĞİLDİR – DURUM TAŞIR

`state.guaranteedTickets` turlar arasında yaşar. İmzayı saf tutmak istersen `(state, ...) => [newState, result]` şeklinde yaz, ama **durumu atma** — sticky seçim bu sistemin bir özelliğidir, bug değil (bkz. §3.4).

BİREBİR EŞLEŞMESİ GEREKENLER

- **Adım sırası:** havuz → budama → aciliyet → bütçe → aktif-önce doldurma → kuyruk doldurma.
- **Aciliyet bütçeden önce** eklenir ve `slotsToFill` negatif olabilir (o zaman ek seçim yok).
- **Poisson bütçe:** `truncatedPoissonSample(slotCount - 1, λ, rng) + 1`. +1'i unutma.
- **Bütçe yalnız aktifleri sayar:** `budget - count(guaranteed n activeSet)`.
- **Adaylar varış sırasında** kalmalı; ağırlık `Math.pow(decay, indexInRemaining)` ve **her çekimde yeniden hesaplanır**.
- **Strict** < aciliyet karşılaştırmasında.
- Yalnız `state === 'Active'` biletler acil olabilir.

`DistributorState` gün/retry değişiminde **sıfırdan kurulmalıdır**.

```
generateNoise(state: DistributorState, upcoming: SimTicket[], settings:
BoardDistributionSettings, rng: Rng): BoardItem[]
```

Unity karşılığı: `BoardDistributor.LeakNoiseItems`

BİREBİR EŞLEŞMESİ GEREKENLER

- **Budama önce:** `state.leakedTickets n upcoming`.
- **Adaylar:** `upcoming.slice(0, leakDepth).filter(not leaked)`. Boşsa **hiç çekim yapmadan** dön.
- **Sayı:** `min(truncatedPoissonSample(maxLeakCount, λ, rng), candidates.length)`.
- **İki aşamalı seçim:** önce uniform bilet (yerine koymadan, `ssplice`), sonra o biletin item'larından uniform bir tane.
- Modifikasyonlar yalnız Main ise taşınır.
- Sızdıran bilet `state.leakedTickets` 'a eklenir.

Çıktı **sıralı bir liste** olmalı — spawn sırası hücre seçimini etkiler.

`generateBoardSpawn(state: SimState, rng: Rng): SpawnResult`

Unity karşılığı: `BoardDistributor.OnOrderPlaced` + `BoardGrid.RequestSpawn`

AKIŞ

```
generateBoardSpawn(state, rng):
    // 1. Gerekli havuz – ÖNCE
    guaranteed = selectGuaranteedTickets(state.dist, state.active, state.upcoming, s, rng)
    missing    = getMissingItems(guaranteed, state.board)
    for (key, n) of sortedEntries(missing):          // §10.2 UNKNOWN
        for i of range(n): requestSpawn(state.board, itemFromKey(key), rng)

    // 2. Gürültü – SONRA
    for item of generateNoise(state.dist, state.upcoming, s, rng):
        requestSpawn(state.board, item, rng)
```

REQUESTSPAWN SÖZLEŞMESİ

- `rng` verilirse: **tüm boş hücreler toplanır** (Y dış, X iç tarama) ve uniform seçilir.
- `rng` verilmezse: ilk boş hücre (aynı tarama sırası).
- Boş hücre yoksa: `pendingSpawns` FIFO kuyruğuna eklenir, `false` döner.
- `removeItem(x,y)` : hücre boşalır, **kuyrukta item varsa aynı hücreye anında backfill** edilir.

Çağırılma koşulu: her `TicketAssigned` 'da – bilet null olsa bile. Açılış bastırma sayacı (`openingAssignmentsWithoutDistribution`) modellenmeli, yoksa açılış tahtası yazan günler yanlış simüle edilir.

10.2 Determinizm kontrol listesi

Simülâtörün Unity ile **eşleşemeyeceği** noktalar, en önemliden aza:

#	KAYNAK	ETKİSİ	ÖNERİ
1	<code>System.Random</code> algoritması	Tam dizi eşleşmesi imkânsız	RNG'yi enjekte et, dağılım karşılaştır
2	<code>Dictionary</code> iterasyon sırası (<code>neededCounts</code>)	Spawn <i>sırası</i> , dolayısıyla hücre yerleşimi	Deterministik sırala, hücre eşleşmesinden vazgeç
3	<code>HashSet.ToList()</code> sırası (<code>guaranteedTickets</code>)	Yok – sonuç bir sayım, sıra bağımsız	Endişelenme
4	<code>float</code> vs JS <code>number</code> (double)	Poisson terimlerinde son basamak	Pratikte ihmal edilebilir
5	<code>Time.deltaTime</code> değişkenliği	Timeout anları	Sabit adımlı tick kullan

10.3 Simülatörün ölçmesi gereken metrikler

Kodun yapısından çıkan, balans için anlamlı ölçütler:

- **Sıfır-tamamlanabilir tur oranı** — hiçbir aktif biletin tamamlanamadığı tur yüzdesi. Kod yorumlarında kayıtlı bir playtest bug'ında bu **%18.7**'ye çıkmış ve D-044 düzeltmesiyle sıfıra inmiş. Bu, sistemin sağlık göstergesidir.
- **Kilitlenme oranı** — üç slot da dolu ve hiçbirini tamamlanamıyor (eski ölçüm: %2.3).
- **Zorunlu can kaybı / gün** (eski ölçüm: ~0.3).
- **Board doluluk eğrisi** ve `pendingSpawns` uzunluğu — item'lar hiç eksilmediği için (§5.5) doyunluk kaçınılmazdır; soru ne zaman.
- **Ortalama kalan süre oranı** teslimat başına → bahşiş kademesi dağılımı → günün beklenen geliri.

§11 UNKNOWN / NEEDS DECISION listesi

Koddan kesin cevabını çıkaramadığım veya senin karar vermen gereken her nokta.

#	KONU	TIP	BÖLÜM
U1	JsonUtility eksik-nesne semantiği . Kod yorumları "zeroed instance, asla null" diyor ama <code>ResolveDay</code> yine de null kontrolü yapıyor. Çalıştırarak doğrulamadım. TS'de normalizasyon katmanı gerekir.	UNKNOWN + DECISION	§1.1
U2	System.Random taşınamaz . Unity seed'ile aynı diziyi üretmek mümkün değil. Doğrulama stratejisi seçilmeli.	DECISION	§2.3
U3	Dictionary iterasyon sırası gerekli-havuz spawn sırasını, dolayısıyla hücre yerleşimini belirsiz bırakıyor.	UNKNOWN + DECISION	§10.1
U4	triggerStepIndex ≠ -1 hiçbir yerden tetiklenmiyor. Kalıntı mı, gelecek özellik mi belli değil. Böyle bir entry sessizce hiç spawn edilmiyor.	UNKNOWN + DECISION	§7.5
U5	dayIndex için üst sınır yok . Parser <code>dayIndex</code> 'i doğrulamaz; negatif veya çok büyük değer kabul edilir ve yalnız sıralamayı etkiler. Yinelenen değer hata basar ama iki gün de listede kalır — hangisinin "kazandığı" <code>CurrentDayIndex</code> pozisyonuna göre değişir.	UNKNOWN	§1.3
U6	ticketsRequiredForDay runtime'da hiç kontrol edilmiyor . Yalnız validator karşılaştırır. Elle düzenlenmiş, uyuşmayan bir dosya sorunsuz yüklenir ve gün dizi tükendiğinde biter.	UNKNOWN	§6.5
U7	Board boyutu Day JSON'da değil . Tüm günler <code>GameConfig</code> 'ten gelen tek bir boyutu paylaşır. Harici editör bu değeri nereden okuyacak — elle mi girilecek, ayrı bir config dosyasına mı yazılacak?	DECISION	§5.1
U8	EconomyConfig ve FoodItemConfig.basePrice ScriptableObject'te . Simülatörün bahşış/gelir hesabı için bu değerleri dışa aktarman gerekir; JSON'da yoklar.	DECISION	§8.4
U9	FoodCatalog da ScriptableObject . Yemek id'leri, kategoriler ve <code>availableModifications</code> listeleri harici editöre bir şekilde aktarılmalı (export adımı yazılmalı).	DECISION	§1.3
U10	Powerup'ların board üzerindeki etkisi (özellikle Noise Clear, <code>NoiseClearRunner</code>) bu dökümanın kapsamı dışında bırakıldı. Balancing simülatörü powerup kullanımını modelleyecekse ayrıca incelenmeli.	UNKNOWN	§5.5
U11	Tutorial'ın board üzerindeki kilitleri (<code>TutorialDirector</code>) incelenmedi. <code>day_00</code> simüle edilecekse gerekir; diğer günlerde tutorial yok.	UNKNOWN	§6.4

DOĞRULANMIŞ SAYILABİLECEKLER

Aşağıdakileri kodda **doğrudan okudum**, çıkarım yapmadım: çekim sıraları (§2.2), Poisson implementasyonu (§2.2), `EarlyTicketWeightDecay` formülü (§3.4), `+1` bütçe kaydırması (§3.4), leak'in iki aşamalı seçimi (§4), hücre tarama sırası (§5.2), backfill davranışı (§5.3), gün bitiş koşulu (§6.5), süre çözümleme zinciri (§8.1), bahşış formülü (§8.4) ve tüm validation kuralları (§9).

Expo the Explorer · Day Runtime Specification · 2026-08-30
Kaynak: Assets/Scripts/{Systems/DaySystem,
Systems/BoardDistribution, Systems/TicketSystem, Systems/TraySystem,
Systems/EconomySystem, Core, Bootstrap} · Assets/Data/DataScripts ·
Assets/Editor